

CPE 434
VLSI Design

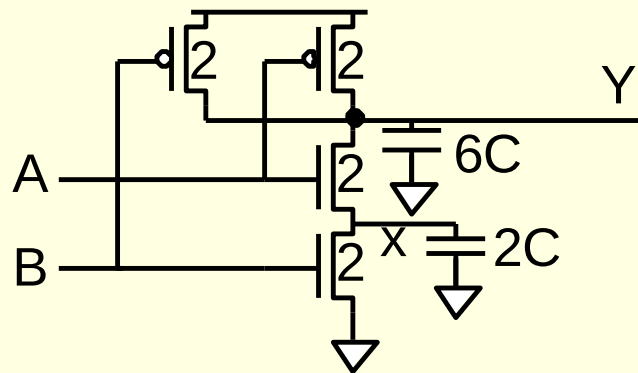
Chapter 9:
Combinational Circuits Design

Optimization Techniques

- Input Ordering Delay Effect
- Asymmetric Gates
- Skewed Gates
- P/N Ratios

Input Ordering Delay Effect

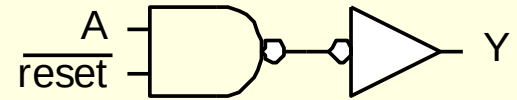
- Calculate parasitic delay for Y falling



- If A arrives latest? 2τ
- If B arrives latest? 2.33τ
- Therefore, if input arrival time is known
 - Connect latest input to inner terminal

Asymmetric Gates

- Asymmetric gates favor one input over another
- Ex: suppose input A of a NAND gate is most critical
 - Use smaller transistor on A (less capacitance)
 - Boost size of noncritical input
 - So total resistance is same

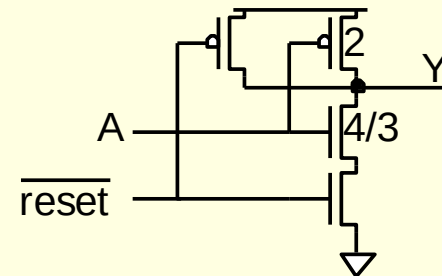


- $g_A =$

- $g_B =$

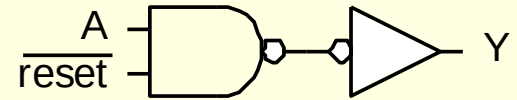
- $g_{\text{total}} = g_A + g_B =$

- Asymmetric gate approaches $g = 1$ on critical input
- But total logical effort goes up



Asymmetric Gates

- Asymmetric gates favor one input over another
- Ex: suppose input A of a NAND gate is most critical
 - Use smaller transistor on A (less capacitance)
 - Boost size of noncritical input
 - So total resistance is same



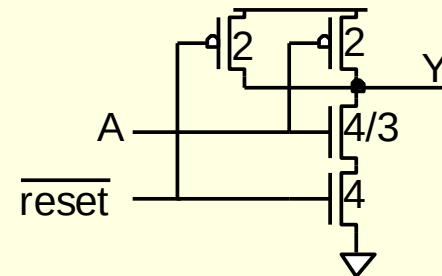
- $g_A = 10/9$

- $g_B = 2$

- $g_{\text{total}} = g_A + g_B = 28/9$

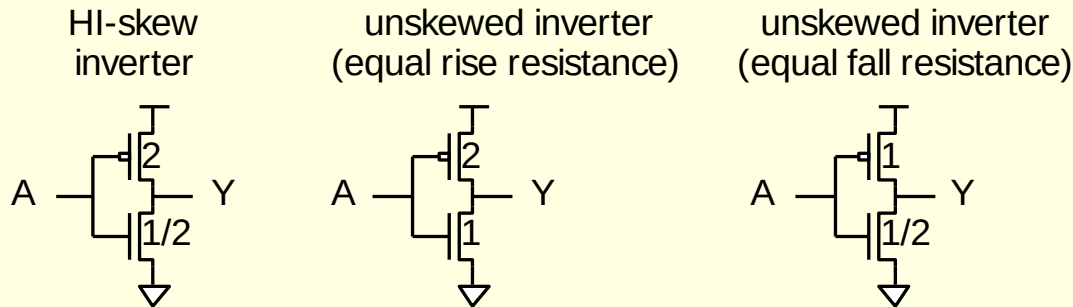
- Asymmetric gate approaches $g = 1$ on critical input

- But total logical effort goes up



Skewed Gates

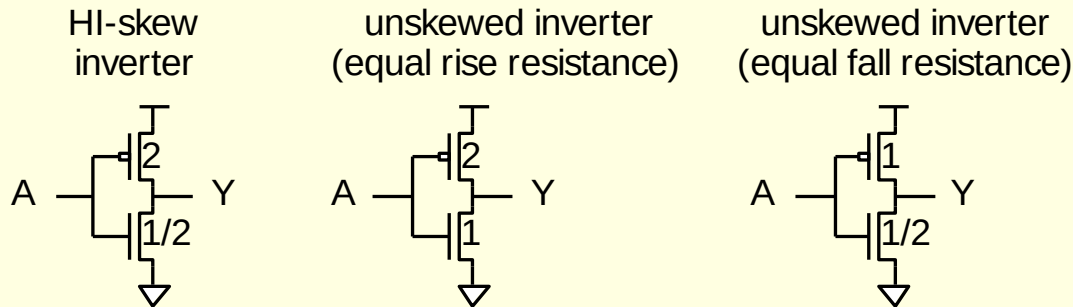
- Skewed gates favor one edge over another
- Ex: suppose rising output of inverter is most critical
 - Downsize noncritical nMOS transistor (with some skewing factor)



- Calculate logical effort by comparing to unskewed inverter with same effective resistance on that edge.
 - $g_u =$
 - $g_d =$

Skewed Gates

- Skewed gates favor one edge over another
- Ex: suppose rising output of inverter is most critical
 - Downsize noncritical nMOS transistor



- Calculate logical effort by comparing to unskewed inverter with same effective resistance on that edge.
 - $g_u = 2.5 / 3 = 5/6$
 - $g_d = 2.5 / 1.5 = 5/3$

HI- and LO-Skew

- Def: Logical effort of a skewed gate for a particular transition is the ratio of the input capacitance of that gate to the input capacitance of an unskewed inverter delivering the same output current for the same transition.
- Skewed gates reduce size of noncritical transistors
 - HI-skew gates favor rising output (small nMOS)
 - LO-skew gates favor falling output (small pMOS)
- Logical effort is smaller for favored direction
- But larger for the other direction

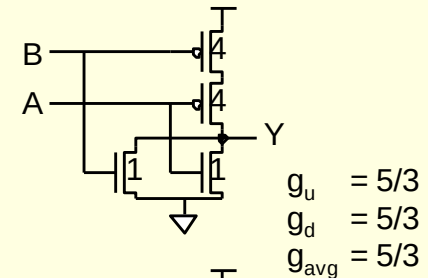
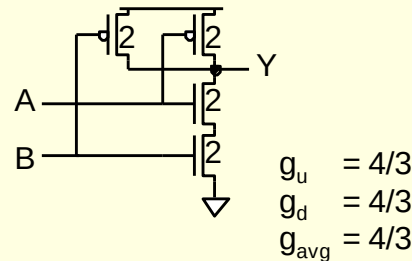
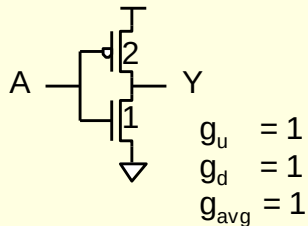
Catalog of Skewed Gates

Inverter

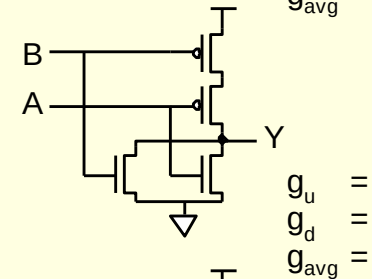
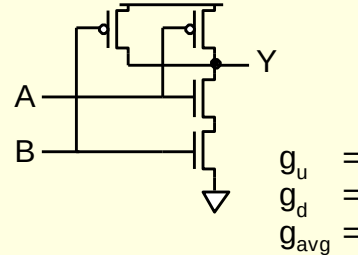
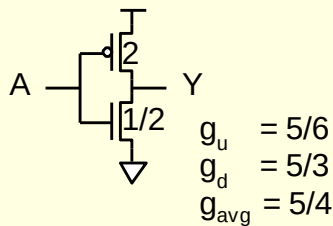
NAND2

NOR2

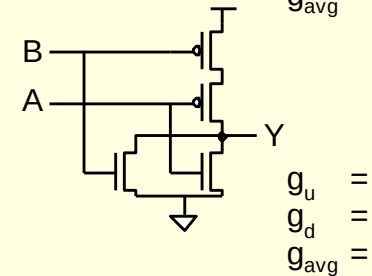
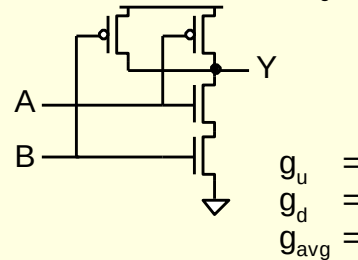
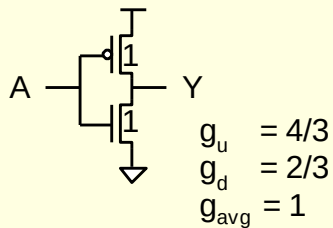
unskewed



HI-skew



LO-skew



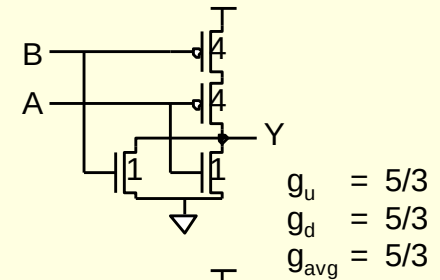
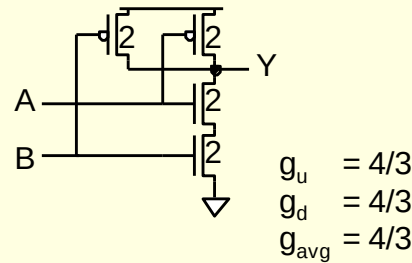
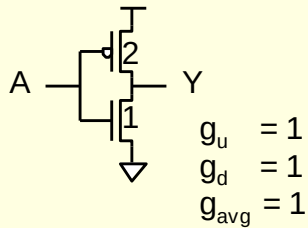
Catalog of Skewed Gates

Inverter

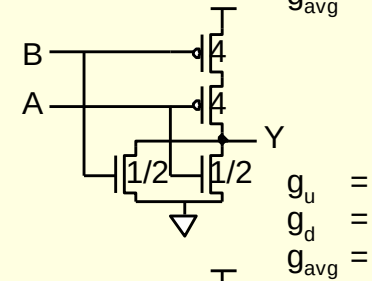
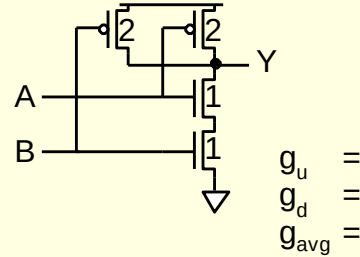
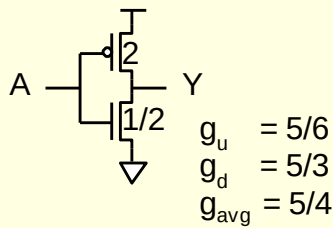
NAND2

NOR2

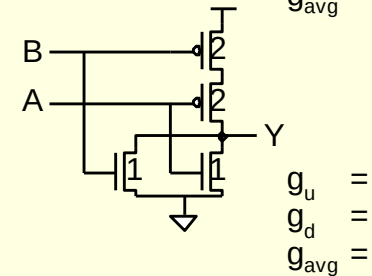
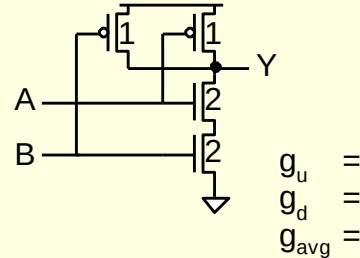
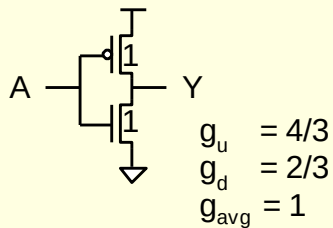
unskewed



HI-skew



LO-skew



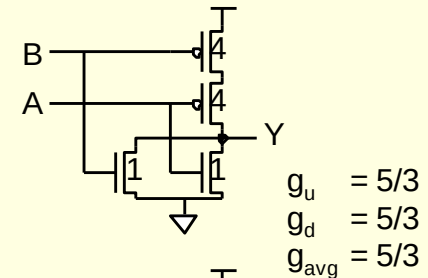
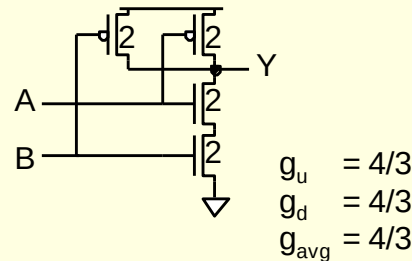
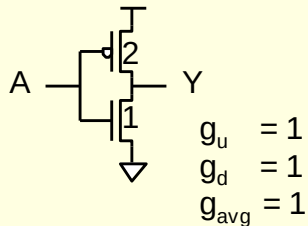
Catalog of Skewed Gates

Inverter

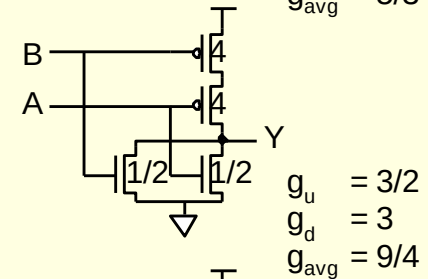
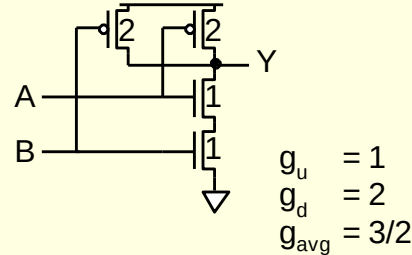
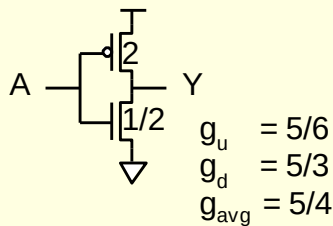
NAND2

NOR2

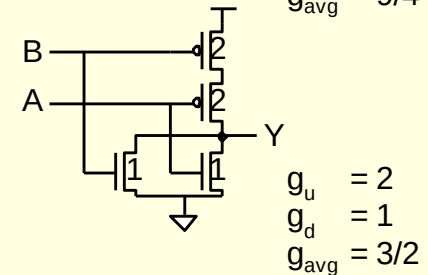
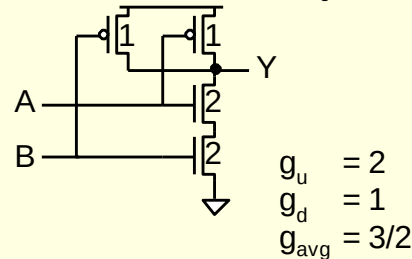
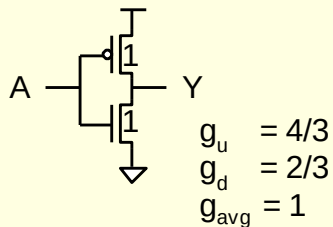
unskewed



HI-skew



LO-skew

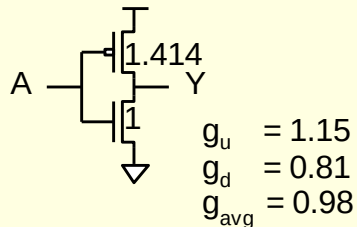


P/N Ratios

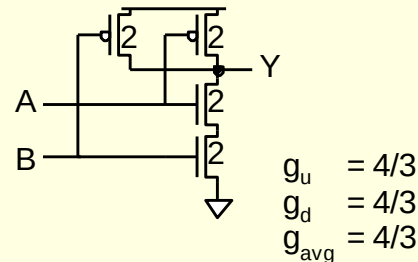
- In general, best P/N ratio is sqrt of equal delay ratio.
 - Only improves average delay slightly for inverters
 - But significantly decreases area and power

fastest
P/N ratio

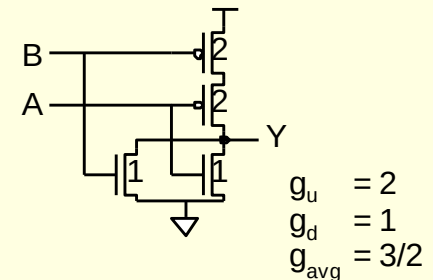
Inverter



NAND2



NOR2

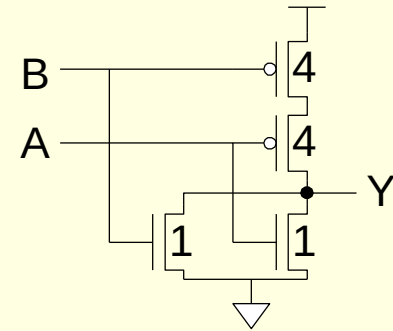


Circuit Families

- Static CMOS Logic
 - Compound
 - NAND
 - NOR
- Ratioed Logic
 - Pseudo-nMOS Logic
- Dynamic Logic
 - Dynamic Logic
 - Domino (Dynamic-Static Pair)
 - Dual-Rail Domino
- Cascode Voltage Switch Logic
- Pass Transistor Logic with Shannon's Expansion Theorem
 - LEAP
 - CPL

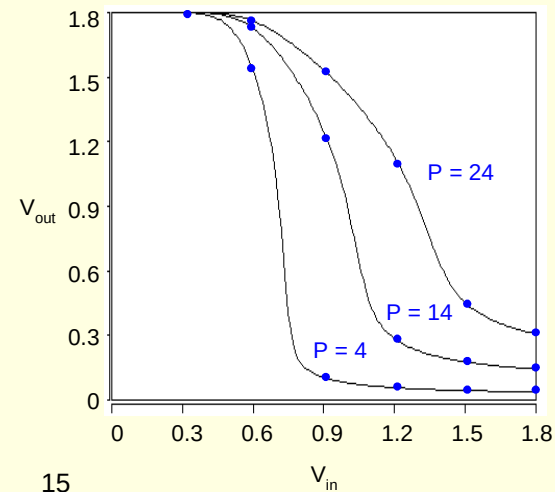
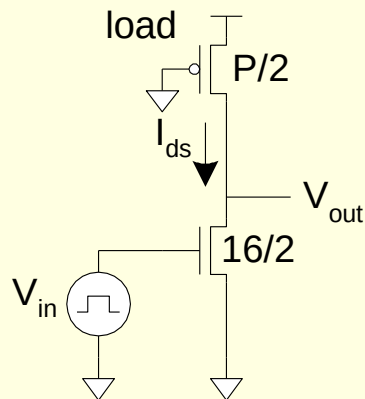
Ratioed Circuit...Introduction

- What makes a circuit fast?
 - $I = C \, dV/dt \rightarrow t_{pd} \propto (C/I) \, \Delta V$
 - low capacitance
 - high current
 - small swing
- Logical effort is proportional to C/I
- pMOS are the enemy!
 - High capacitance for a given current
- Can we take the pMOS capacitance off the input?
- Various circuit families try to do this...



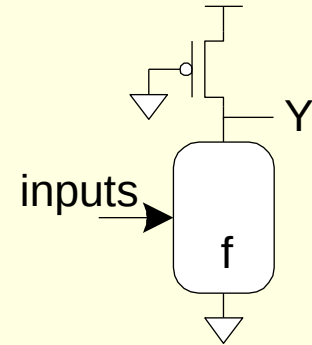
Pseudo-nMOS

- In the old days, nMOS processes had no pMOS
 - Instead, use pull-up transistor that is always ON
- In CMOS, use a pMOS that is always ON
 - *Ratio* issue
 - Make pMOS about $\frac{1}{4}$ effective strength of pulldown network

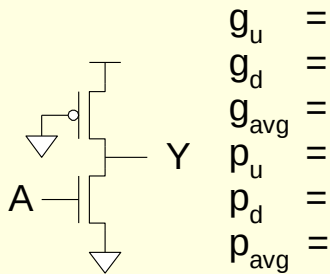


Pseudo-nMOS Gates

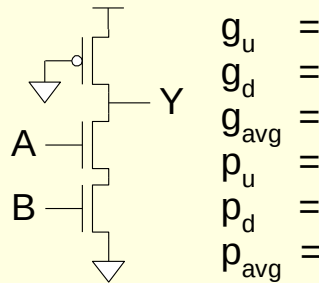
- Design for *unit current* on output to compare with unit inverter.
- pMOS fights nMOS



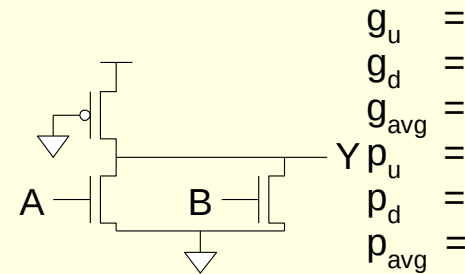
Inverter



NAND2

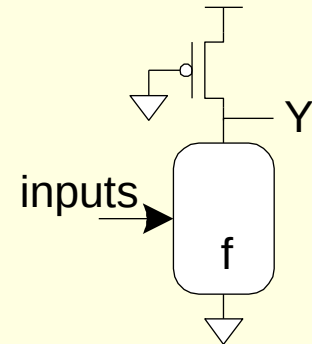


NOR2

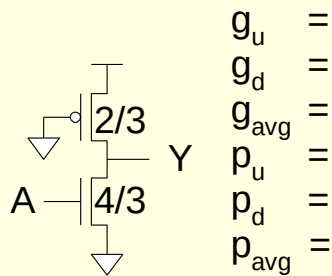


Pseudo-nMOS Gates

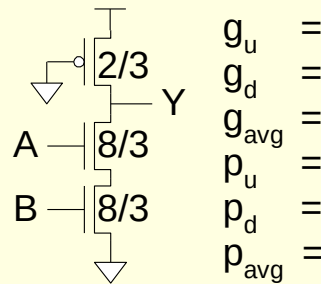
- Design for unit current on output to compare with unit inverter.
- pMOS fights nMOS



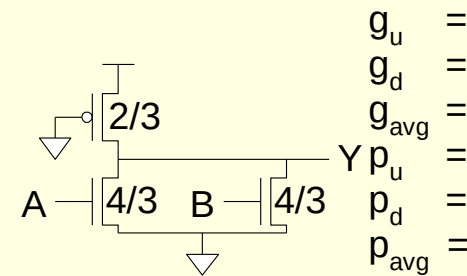
Inverter



NAND2

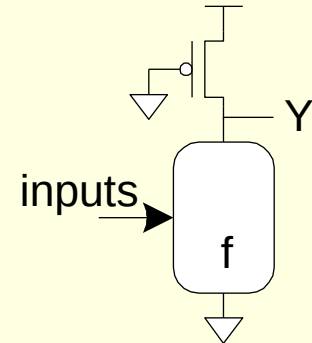


NOR2

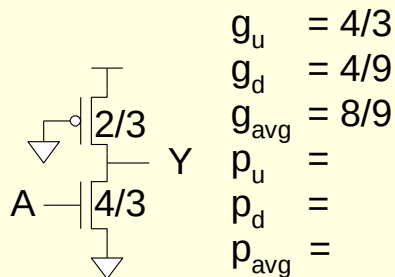


Pseudo-nMOS Gates

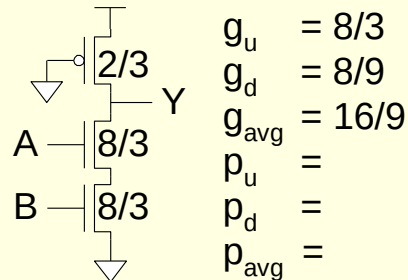
- Design for unit current on output to compare with unit inverter.
- pMOS fights nMOS



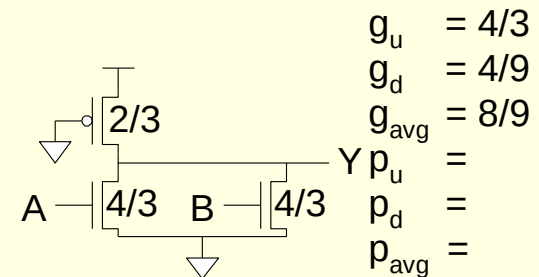
Inverter



NAND2

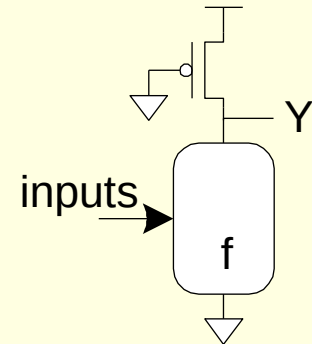


NOR2

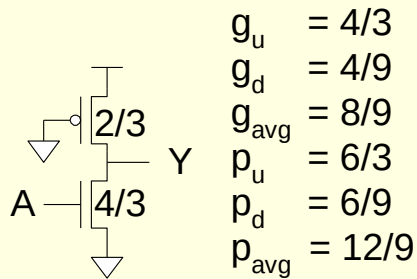


Pseudo-nMOS Gates

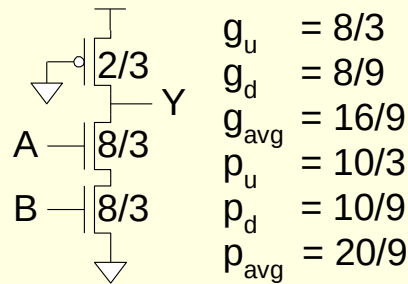
- Design for unit current on output to compare with unit inverter.
- pMOS fights nMOS



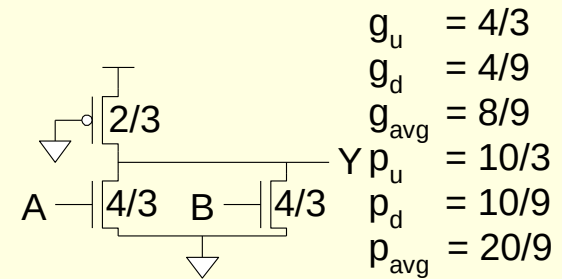
Inverter



NAND2



NOR2



Pseudo-nMOS Design

- Ex: Design a k-input AND gate with DeMorgan's law using static CMOS inverter followed by a k-input pseudo-nMOS NOR. Estimate the delay driving a fanout of H

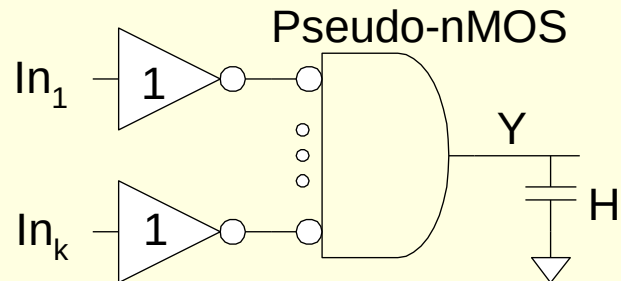
- $G =$

- $F =$

- $P =$

- $N =$

- $D =$



Pseudo-nMOS Design

- Ex: Design a k-input AND gate with DeMorgan's law using static CMOS inverter followed by a k-input pseudo-nMOS NOR. Estimate the delay driving a fanout of H

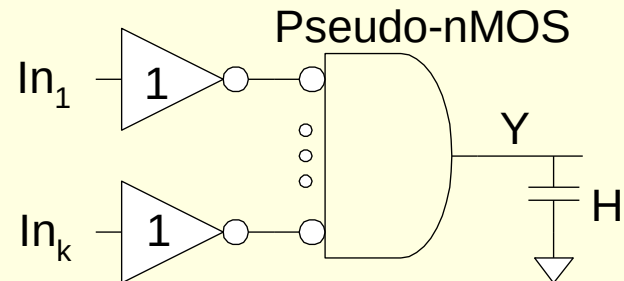
- $G = 1 * 8/9 = 8/9$

- $F = GBH = 8H/9$

- $P = 1 + (4+8k)/9 = (8k+13)/9$

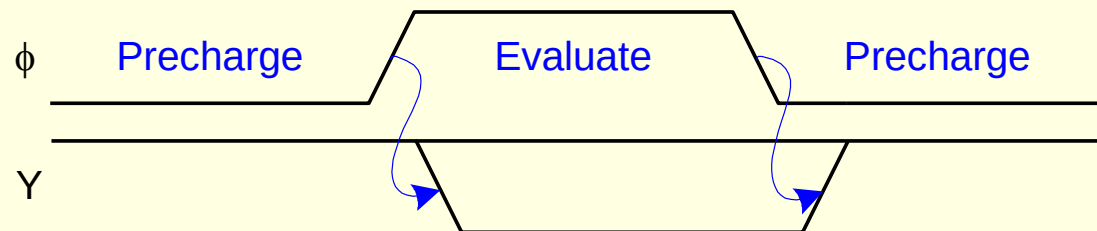
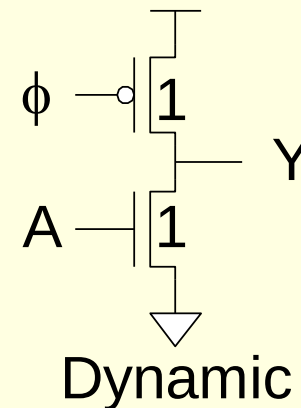
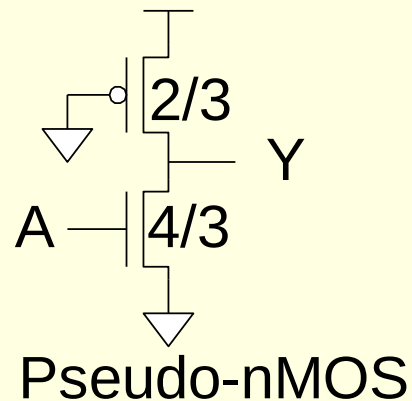
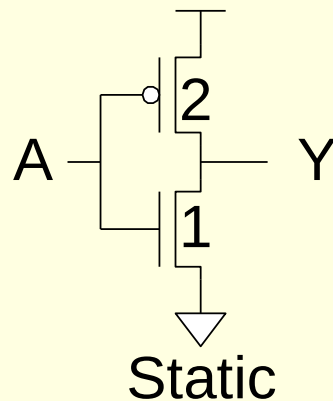
- $N = 2$

- $D = NF^{1/N} + P = \frac{4\sqrt{2H}}{3} + \frac{8k+13}{9}$



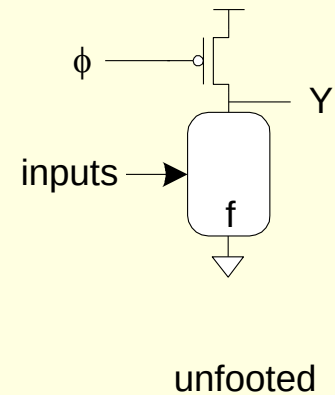
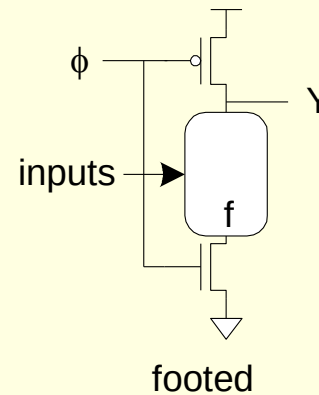
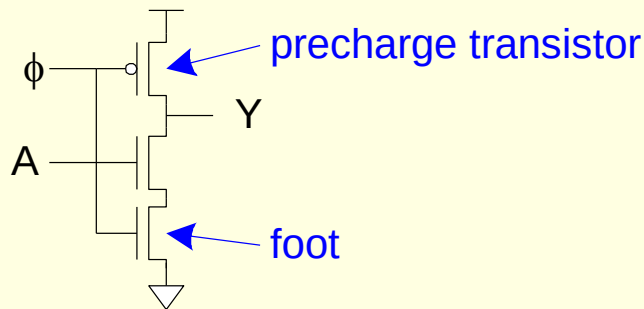
Dynamic Logic

- *Dynamic* gates uses a clocked pMOS pullup
- Two modes: *precharge* and *evaluate*



The Foot

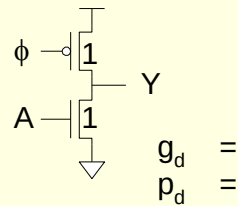
- What if pulldown network is ON during precharge?
- Use series evaluation transistor to prevent fight.



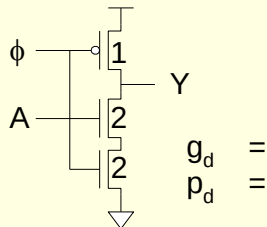
Logical Effort

Inverter

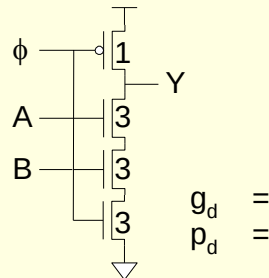
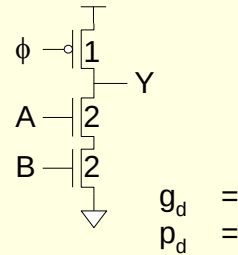
unfooted



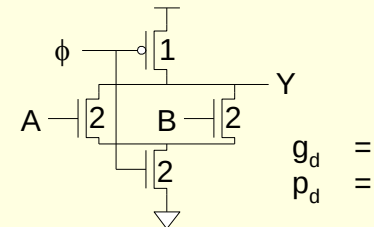
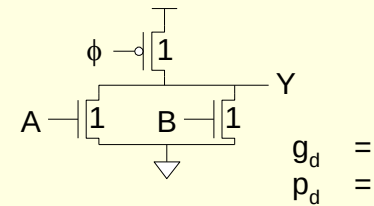
footed



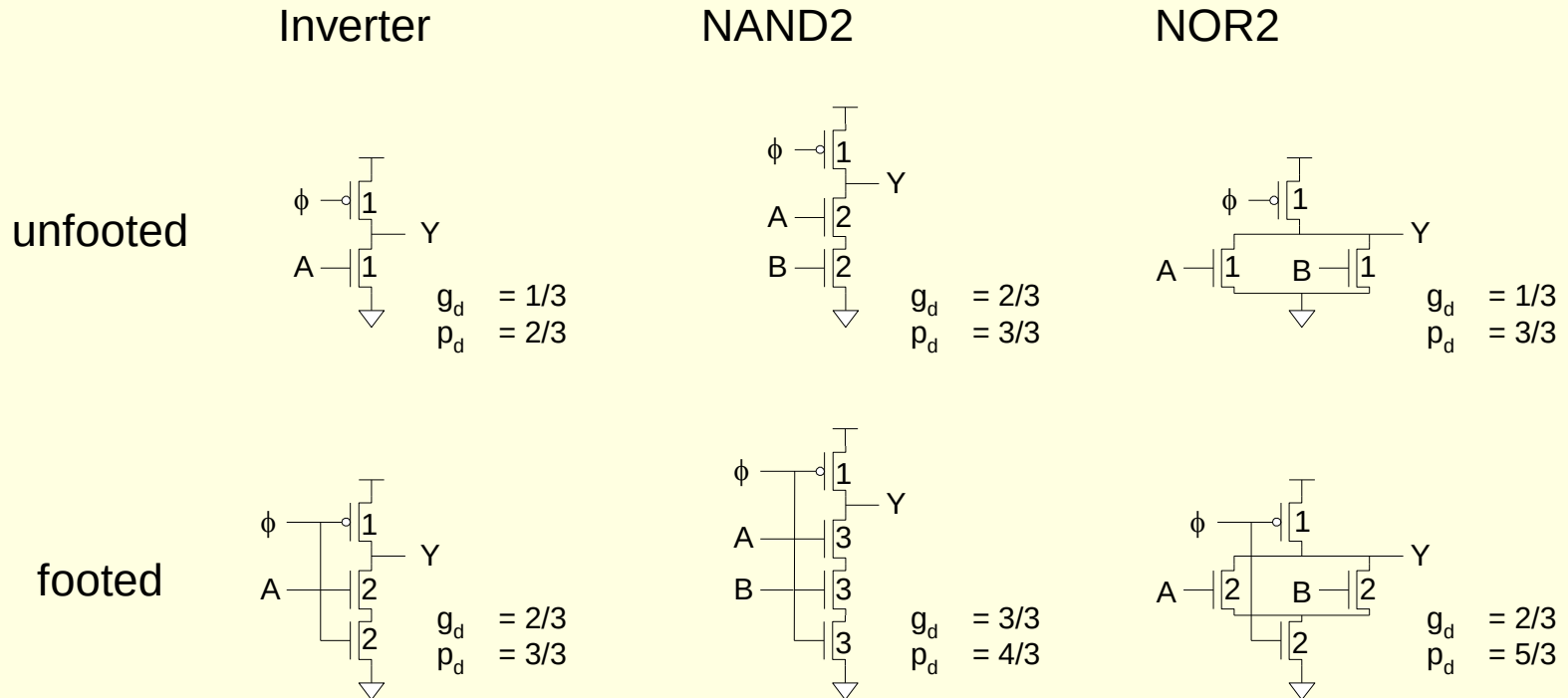
NAND2



NOR2



Logical Effort



Monotonicity

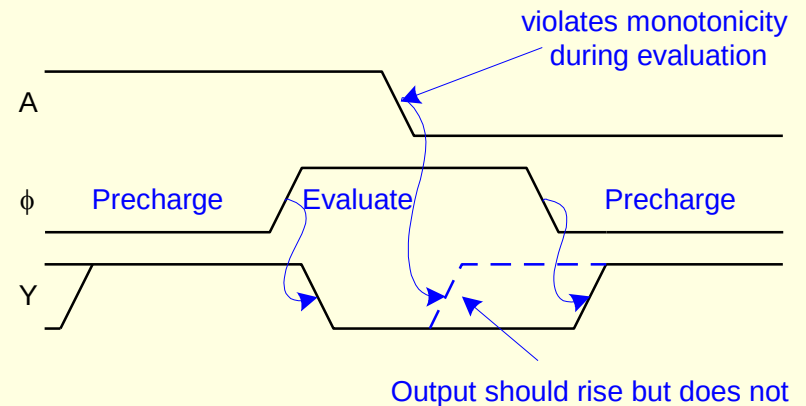
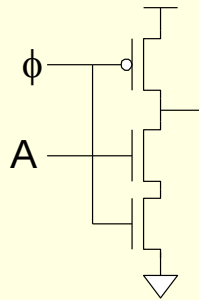
- Dynamic gates require *monotonically rising* inputs during evaluation

- 0 -> 0

- 0 -> 1

- 1 -> 1

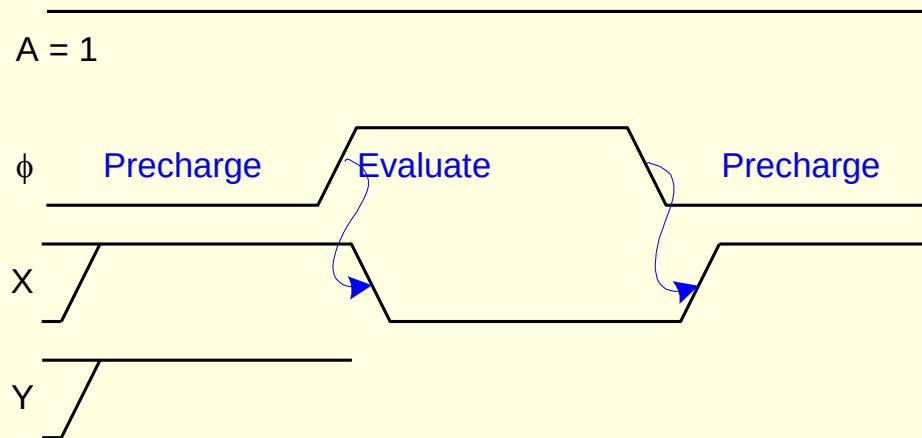
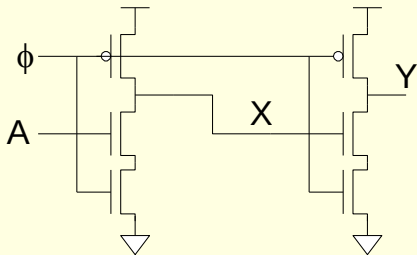
- But not 1 -> 0



- The output will remain low until the next precharge step

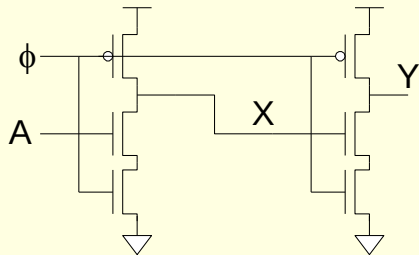
Monotonicity Woes

- But dynamic gates produce monotonically falling outputs during evaluation
- Illegal for one dynamic gate to drive another!

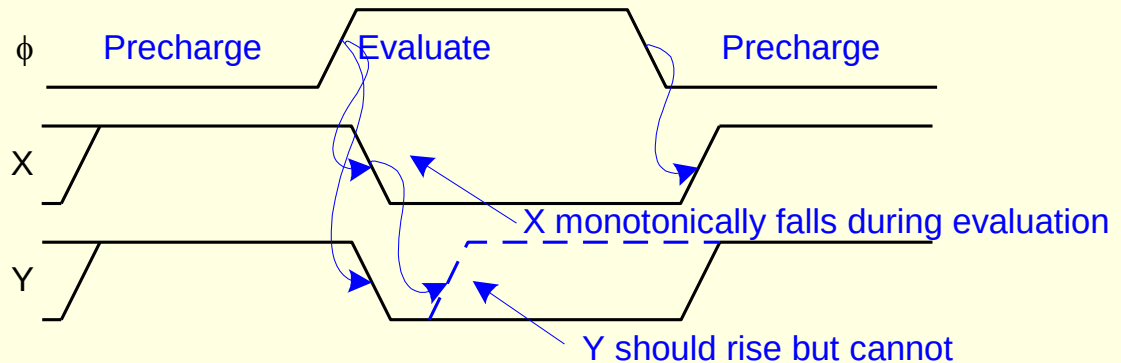


Monotonicity Woes

- But dynamic gates produce monotonically falling outputs during evaluation
- Illegal for one dynamic gate to drive another!

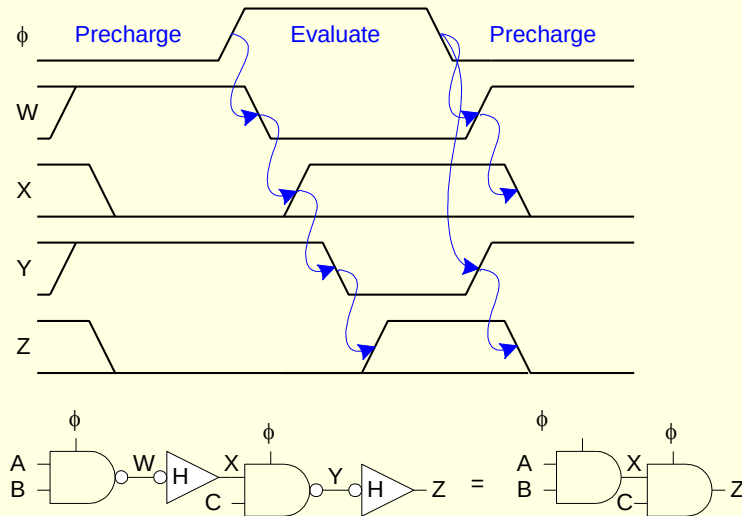
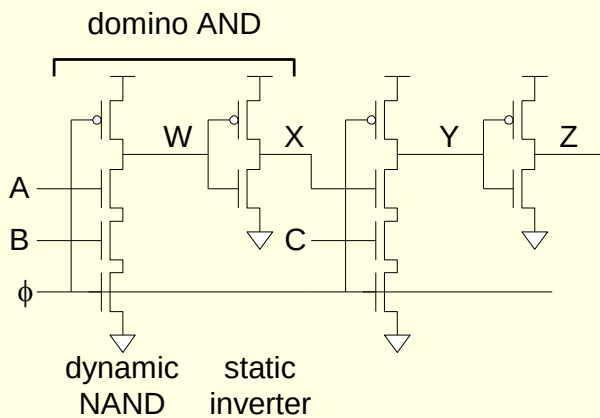


A = 1



Domino Gates

- Follow dynamic stage with inverting static gate
 - Dynamic / static pair is called domino gate
 - Produces monotonic outputs

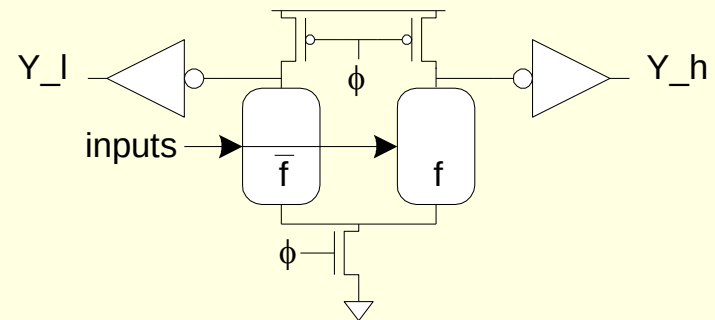


- Domino gates are inherently noninverting

Dual-Rail Domino

- Domino only performs noninverting functions:
 - AND, OR but not NAND, NOR, or XOR
- Dual-rail domino solves this problem
 - Takes true and complementary inputs
 - Produces true and complementary outputs

sig_h	sig_l	Meaning
0	0	Precharged
0	1	'0'
1	0	'1'
1	1	invalid

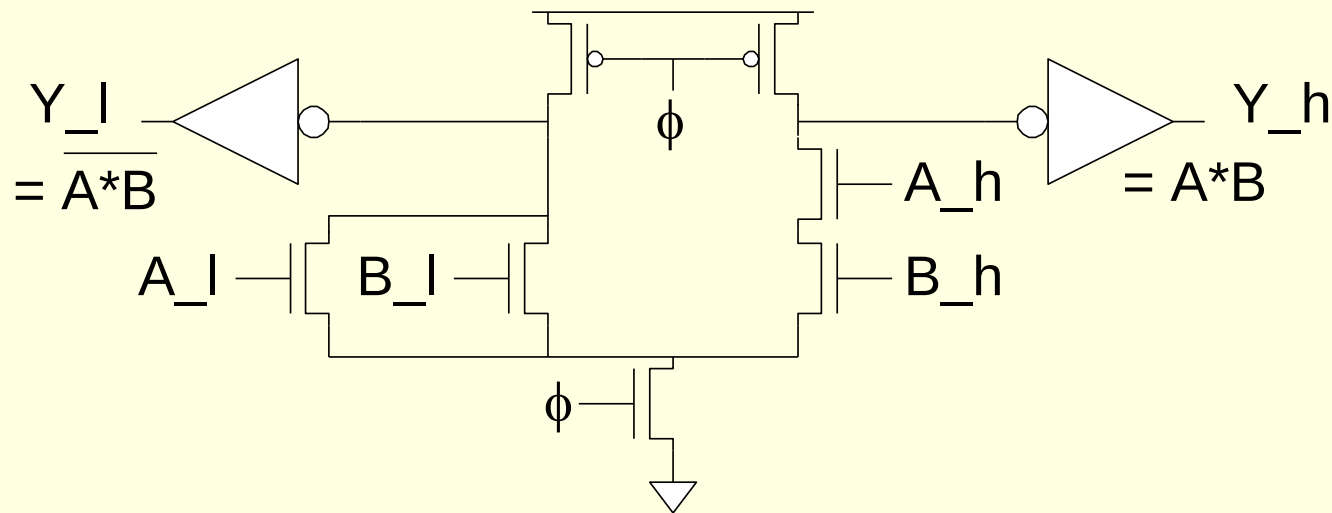


Example: AND/NAND

- Given A_h, A_l, B_h, B_l
- Compute $Y_h = A * B, Y_l = \sim(A * B)$

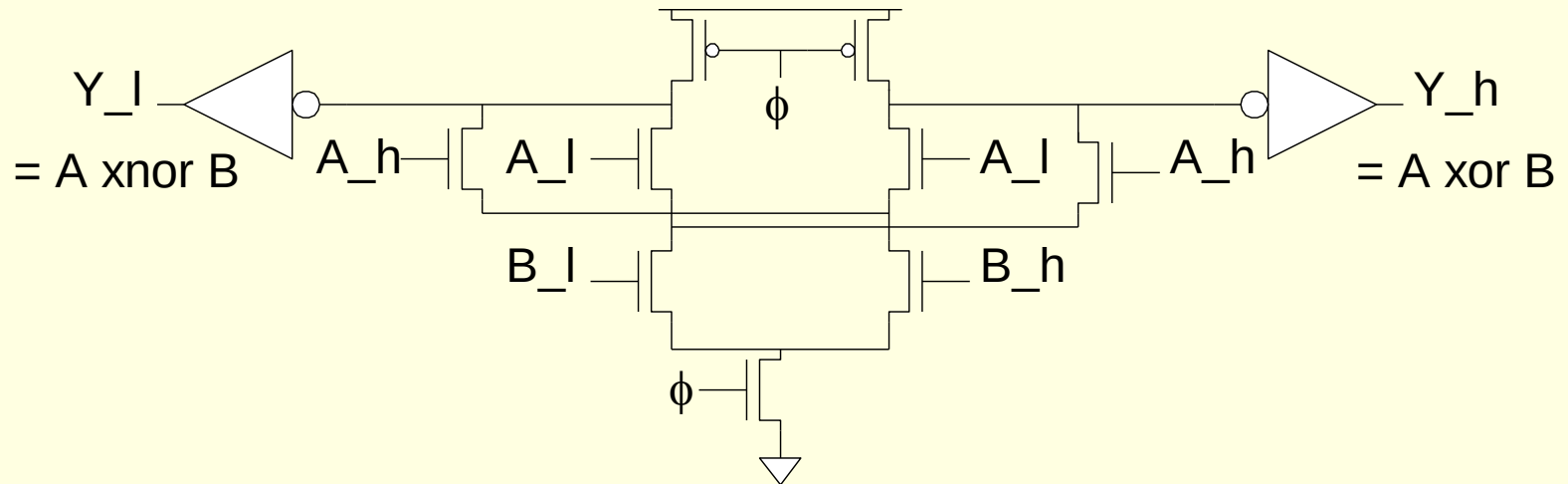
Example: AND/NAND

- Given A_h, A_l, B_h, B_l
- Compute $Y_h = A * B, Y_l = \sim(A * B)$
- Pulldown networks are conduction complements



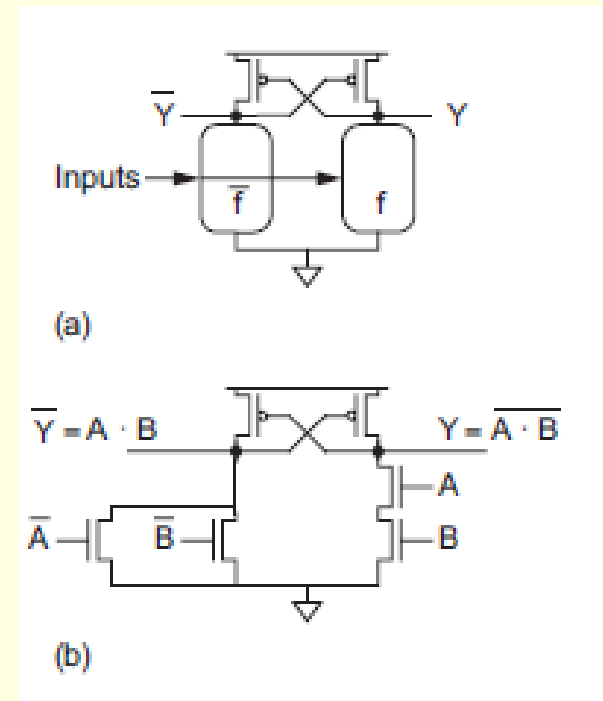
Example: XOR/XNOR

- Sometimes possible to share transistors



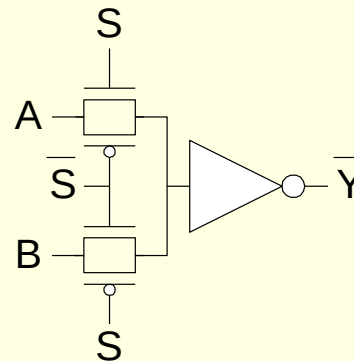
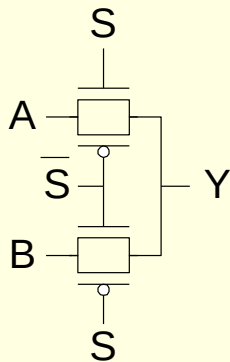
Cascode Voltage Switch Logic

- The pulldown network f implements the logic function as in a static CMOS gate, while f' (f -complement) uses inverted inputs feeding transistors arranged in the conduction complement.
- CVSL of AND/NAND-gate



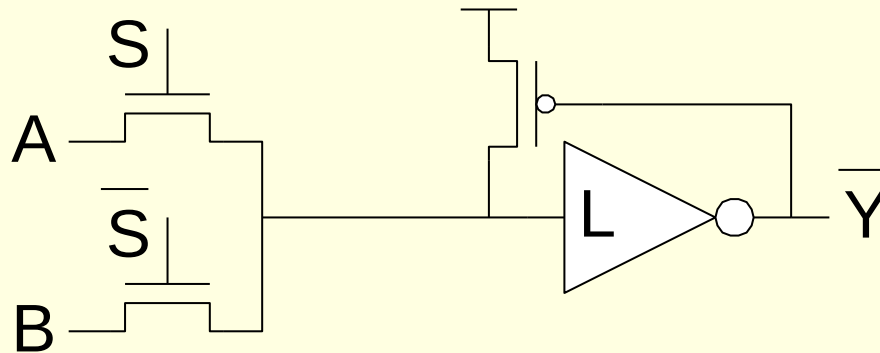
Pass Transistor Circuits

- Use pass transistors like switches to do logic
- Inputs drive diffusion terminals as well as gates
- 2x1 MUX $\rightarrow Y=SA+S'B$
- CMOS + Transmission Gates:
 - 2-input multiplexer
 - Gates should be restoring



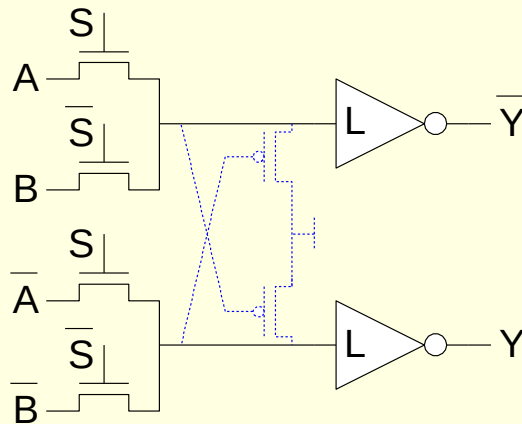
LEAP

- **LEA**n integration with **P**ass transistors
- Get rid of pMOS transistors
 - Use weak pMOS feedback to pull fully high
 - Ratio constraint

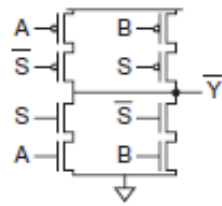


CPL

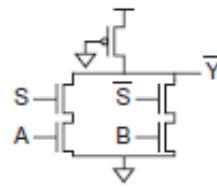
- **C**omplementary **P**ass-transistor **L**ogic
 - Dual-rail form of pass transistor logic
 - Avoids need for ratioed feedback
 - Optional cross-coupling for rail-to-rail swing



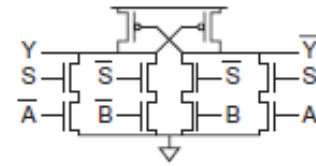
Static CMOS



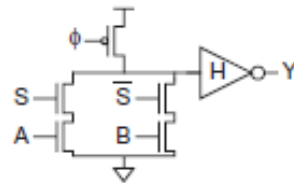
Pseudo-nMOS



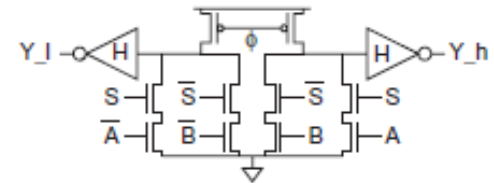
CVSL



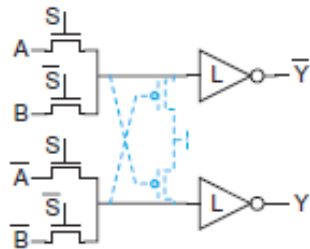
Domino



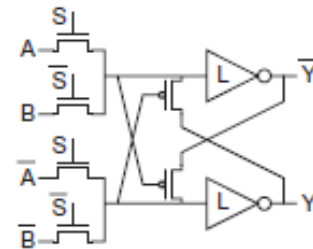
Dual-Rail Domino



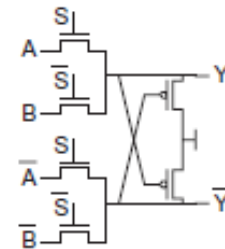
CPL



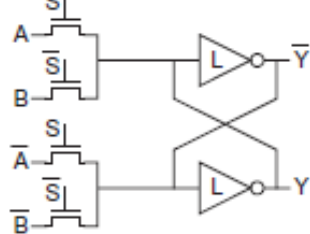
EEPL



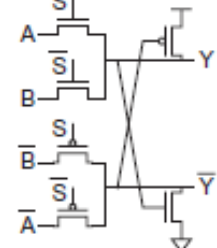
DCVSPG



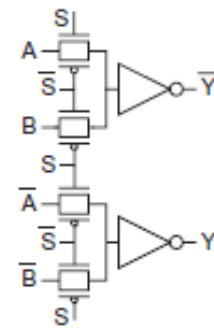
SRPL



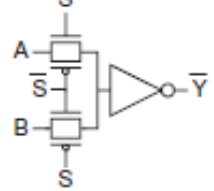
PPL



DPL



CMOSTG



LEAP

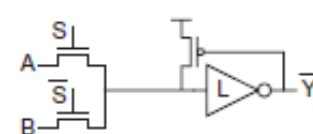


FIGURE 9.47 Comparison of circuit families for 2-input multiplexers

Shannon's Expansion Theorem

- Any Boolean function $f(w_1, w_2, \dots, w_n)$ can be written in the form

$$f(w_1, w_2, \dots, w_n) = \bar{w}_1 \cdot f(0, w_2, \dots, w_n) + w_1 \cdot f(1, w_2, \dots, w_n)$$

- or

$$f(w_1, w_2, \dots, w_n) = \bar{w}_1 \cdot f_{\bar{w}_1} + w_1 \cdot f_{w_1}$$

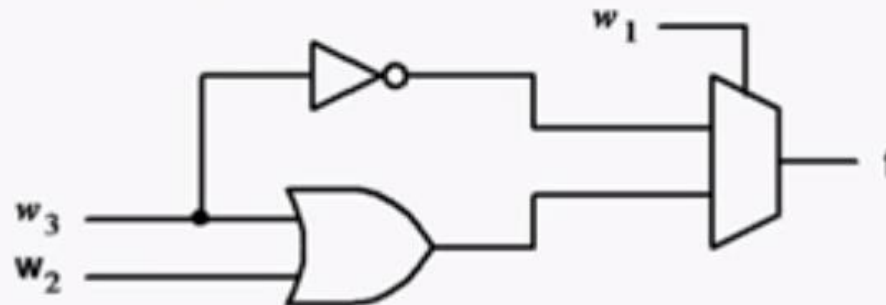
- where $f_{\bar{w}_1}$ is called the cofactor of f with respect to $\bar{w}_1$ and f_{w_1} is called the cofactor of f with respect to w_1
- Shannon's Expansion expresses a given logic function in terms of select inputs and data inputs required for logic synthesis using a multiplexer

Example # 1

- Implement $f = \bar{w}_1\bar{w}_3 + w_1w_2 + w_1w_3$ using a 2-to-1 multiplexer
- Shannon's expansion using w_1 gives

$$\begin{aligned}f &= \bar{w}_1 f(0, w_2, w_3) + w_1 f(1, w_2, w_3) \\&= \bar{w}_1 (1 \cdot \bar{w}_3 + 0 \cdot w_2 + 0 \cdot w_3) + w_1 (0 \cdot \bar{w}_3 + 1 \cdot w_2 + 1 \cdot w_3) \\&= \bar{w}_1 (\bar{w}_3) + w_1 (w_2 + w_3)\end{aligned}$$

- The corresponding circuit is shown below



(a) Using a 2-to-1 multiplexer

Example # 2

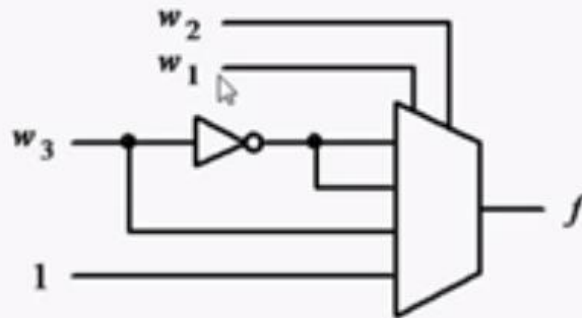
- Implement $f = \bar{w}_1\bar{w}_3 + w_1w_2 + w_1w_3$ using a 4-to-1 multiplexer
- Shannon's expansion using w_1 and w_2 gives

$$\begin{aligned} f &= \bar{w}_1\bar{w}_2f(0,0,w_3) + \bar{w}_1w_2f(0,1,w_3) + \\ &\quad w_1\bar{w}_2f(1,0,w_3) + w_1w_2f(1,1,w_3) \\ &= \bar{w}_1\bar{w}_2(1\cdot\bar{w}_3 + 0\cdot 0 + 0\cdot w_3) + \\ &\quad \bar{w}_1w_2(1\cdot\bar{w}_3 + 0\cdot 1 + 0\cdot w_3) + \\ &\quad w_1\bar{w}_2(0\cdot\bar{w}_3 + 1\cdot 0 + 1\cdot w_3) + \\ &\quad w_1w_2(0\cdot\bar{w}_3 + 1\cdot 1 + 1\cdot w_3) \\ &= \bar{w}_1\bar{w}_2(\bar{w}_3) + \bar{w}_1w_2(\bar{w}_3) + w_1\bar{w}_2(w_3) + w_1w_2(1) \end{aligned}$$

- Shannon's expansion using w_1 and w_2 gives

$$f = \overline{w_1}\overline{w_2}(\overline{w_3}) + \overline{w_1}w_2(\overline{w_3}) + w_1\overline{w_2}(w_3) + w_1w_2(1)$$

- The corresponding circuit is shown below



(b) Using a 4-to-1 multiplexer

Example # 3

- Implement the three-input majority function using only 2-to-1 muxes

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

- Shannon's expansion using w_1 gives

$$\begin{aligned} f &= \bar{w}_1(0 \cdot w_2 + 0 \cdot w_3 + w_2 w_3) + w_1(1 \cdot w_2 + 1 \cdot w_3 + w_2 w_3) \\ &= \bar{w}_1(w_2 w_3) + w_1(w_2 + w_3 + w_2 w_3) \\ &= \bar{w}_1(w_2 w_3) + w_1(w_2 + w_3) \end{aligned}$$

- Let $g = w_2 w_3$ and $h = w_2 + w_3$ and expand both using w_2

$$\begin{aligned} g &= \bar{w}_2(0 \cdot w_3) + w_2(1 \cdot w_3) & h &= \bar{w}_2(0 + w_3) + w_2(1 + w_3) \\ &= \bar{w}_2(0) + w_2(w_3) & &= \bar{w}_2(w_3) + w_2(1) \end{aligned}$$